

République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

École nationale d'électronique
et des télécommunications de Sfax



Ingénieur en :
Génie des Télécommunications

Stage D'été

présenté à

**L'École Nationale d'Électronique et des
Télécommunications de Sfax**

par

Hosni Nheri

**Agent DevOps Autonome pour Infrastructure Auto-
Réparatrice**

M. Abdelkhalek Bakkari

Encadrant



DEDICATION

À mes chers parents, qui m'ont toujours soutenu et cru en moi, je dédie ce travail.

À ma sœur et à toute ma famille, pour leur amour et leurs encouragements permanents.

À mes professeurs et à tous ceux qui transmettent le savoir avec passion.

À mes amis et collègues de promotion, pour les moments partagés et l'entraide tout au long de mon parcours.

À vous tous,

Je dédie ce travail.

Hosni Nheri



REMERCIEMENT

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage et de ce rapport.

Tout d'abord, je remercie chaleureusement mon encadrant professionnel, M. Abdelkhalek Bakkari, pour m'avoir accueilli au sein de Smartovate LTD, pour sa disponibilité, ses conseils techniques précieux et la confiance qu'il m'a accordée tout au long de cette période.

Je remercie les membres de l'équipe pour leur accueil, leur esprit d'équipe et les connaissances qu'ils ont partagées avec moi au quotidien.

Un grand merci à ma famille pour son soutien moral et matériel indéfectible, ainsi qu'à mes amis pour leur encouragement permanent.

Enfin, je remercie l'ensemble du corps enseignant de l'ENET'COM Sfax pour la qualité de la formation dispensée, qui m'a donné les bases nécessaires pour mener à bien ce projet.



TABLE DES MATIÈRES

LISTE DES FIGURES	iv
LISTE DES TABLEAUX	v
LISTE DES ABRÉVIATIONS	vi
INTRODUCTION GÉNÉRALE	vii
1 CONTEXTE ET ANALYSE DES BESOINS	1
1.1 INTRODUCTION	2
1.2 CONTEXTE DU PROJET	2
1.3 PROBLÉMATIQUE	2
1.4 ÉTUDE DE L'EXISTANT	3
1.5 CRITIQUE DE L'EXISTANT	4
1.6 SOLUTION PROPOSÉE	4
1.7 MÉTHODOLOGIE ADOPTÉE	7
1.8 CONCLUSION	7
2 Étude conceptuelle	8
2.1 INTRODUCTION	9
2.2 IDENTIFICATIONS DES ACTEURS	9
2.3 IDENTIFICATIONS DES BESOINS	9
2.3.1 BESOINS FONCTIONNELS	9
2.3.2 BESOINS NON FONCTIONNELS	10
2.4 DIAGRAMME DE CAS D'UTILISATION	11
2.5 DIAGRAMMES DE SÉQUENCE	12
2.5.1 SCÉNARIO CLÉ : DÉTECTION ET RÉMEDIATION D'UN INCIDENT	12
2.5.2 SCÉNARIO CLÉ : BLOCAGE ANTI-FLAPPING	12
2.6 DIAGRAMME DE CLASSES	13
2.7 CYCLE DE VIE D'UNE ALERTE	14

2.8	CONCEPTION DE L'ARCHITECTURE TECHNIQUE	14
2.8.1	ARCHITECTURE RÉSEAU	14
2.8.2	FLUX DE DONNÉES END-TO-END	15
2.9	CONCLUSION	16
3	Réalisation et résultats	17
3.1	INTRODUCTION	18
3.2	ENVIRONNEMENT LOGICIEL	18
3.3	IMPLÉMENTATION	20
3.3.1	STRUCTURE DU PROJET	20
3.3.2	DÉPLOIEMENT DE L'INFRASTRUCTURE AVEC TERRAFORM	20
3.3.3	SURVEILLANCE : CloudWATCH ET SNS	21
3.3.4	FONCTION LAMBDA INGESTION	22
3.3.5	FONCTION LAMBDA REMÉDIATION ET DIAGNOSTIC	23
3.3.6	VERROU ANTI-FLAPPING AVEC DynamoDB	24
3.3.7	REMÉDIATION : DOCUMENTS ET COMMANDES SSM	25
3.3.8	TESTS END-TO-END ET VALIDATION	27
3.4	TESTS ET VALIDATION	29
3.5	DIFFICULTÉS RENCONTRÉES ET SOLUTIONS	30
3.6	CONCLUSION	30
	BIBLIOGRAPHIE	30
	ANNEXES	32
3.7	VUE GLOBALE ET RÉSEAU	33
3.8	INSTANCES EC2	34
3.9	TESTS DES FONCTIONS LAMBDA	35
3.10	DÉPLOIEMENTS TERRAFORM	36

LISTE DES FIGURES

1.1	Carte mentale des composants de l'agent DevOps	6
1.2	Architecture générale de la solution	6
2.1	Diagramme de cas d'utilisation global	11
2.2	Diagramme de séquence – Détection et remédiation d'un incident	12
2.3	Diagramme de séquence – Blocage anti-flapping	13
2.4	Diagramme de classes du système	13
2.5	Cycle de vie d'une alerte CloudWatch	14
2.6	Architecture réseau du VPC 10.0.0.0/16	15
2.7	Flux de données end-to-end entre les services AWS	15
3.1	Arborescence du projet	20
3.2	Console CloudWatch – alarme CPU élevé en état ALARM	21
3.3	Rubrique SNS agent-devops-alarms (type Standard, ARN us-east-1)	22
3.4	Fonction Lambda Ingestion avec son déclencheur SNS et son ARN	22
3.5	Logs CloudWatch – réception réelle de l'alarme SNS « agent-devops-cpu-high »	23
3.6	Fonction Lambda Remediation présentant son code	24
3.7	Table DynamoDB remediation-locks (Active, TTL activé)	25
3.8	Exemple de verrou anti-flapping créé par la Lambda Remediation	25
3.9	Documents SSM (restart-service et scale-up)	26
3.10	Document SSM agent-devops-rollback via CLI (statut Active)	26
3.11	Commande SSM exécutée avec succès sur 2 instances cibles	26
3.12	Sortie SSM confirmant nginx Active : active (running)	27
3.13	Sortie de terraform state list – 39 ressources gérées	28
3.14	ASG avec Desired Capacity = 3 après remédiation automatique	28
3.15	Réponse HTTP 429 – mécanisme anti-flapping	28
3.16	Console IAM – création de l'utilisateur dédié au projet (A.1)	32
3.17	Utilisateur IAM agent-devops-user – clé d'accès générée (A.2)	33
3.18	AWS Global View – récapitulatif des ressources déployées (A.3)	33

3.19 Console VPC – agent-devops-vpc disponible (A.4)	34
3.20 Console EC2 – 3 instances web en état running (A.5)	34
3.21 CLI – instances EC2 visibles avec statut SSM Online (A.6)	34
3.22 Test de la Lambda ingestion – alerte reçue (200) (A.7)	35
3.23 Diagnostic automatique – CPU_OVERLOAD, confiance HIGH (A.8)	35
3.24 Test de la Lambda remediation – action scale_up_asg (A.9)	36
3.25 Terraform apply – déploiement des 7 ressources réseau (A.10)	36
3.26 Terraform apply – déploiement de la Lambda ingestion (A.11)	37
3.27 Terraform apply – déploiement de la table DynamoDB (A.12)	37



LISTE DES TABLEAUX

1.1	Étude comparative des solutions existantes	3
1.2	Planification des sprints	7
2.1	Identification des acteurs	9
2.2	Besoins fonctionnels	10
2.3	Besoins non fonctionnels	11
3.1	Récapitulatif des outils et versions utilisés	19
3.2	Ressources AWS déployées par Terraform	21
3.3	Tests fonctionnels	29
3.4	Difficultés rencontrées et solutions	30



LISTE DES ABRÉVIATIONS

AWS	Amazon Web Services
IaC	Infrastructure as Code
VPC	Virtual Private Cloud
EC2	Elastic Compute Cloud
ASG	Auto Scaling Group
ALB	Application Load Balancer
SNS	Simple Notification Service
SSM	Systems Manager
TTL	Time To Live
IAM	Identity and Access Management
MFA	Multi-Factor Authentication
CPU	Central Processing Unit
ARN	Amazon Resource Name
CLI	Command Line Interface
HTTP	HyperText Transfer Protocol



INTRODUCTION GÉNÉRALE

La disponibilité des services informatiques est devenue un enjeu critique pour les entreprises modernes. Les incidents d'infrastructure — surcharge CPU, panne de service, échec de déploiement — peuvent entraîner des interruptions de service coûteuses lorsque leur détection et leur résolution dépendent exclusivement de l'intervention manuelle des équipes d'exploitation. Le délai de remédiation est un facteur clé : chaque minute d'indisponibilité se traduit par une perte financière et une dégradation de la confiance des utilisateurs.

Dans ce contexte s'inscrit le présent stage d'été, effectué au sein de Smartovate LTD, dont l'objectif est de concevoir et de mettre en œuvre un agent DevOps autonome capable de détecter automatiquement les incidents sur une infrastructure cloud et de les remédier sans intervention humaine, tout en évitant les actions répétitives nocives grâce à un mécanisme de verrouillage.

La solution développée, un **agent DevOps autonome pour infrastructure auto-réparatrice**, s'appuie sur les services d'Amazon Web Services (AWS) : les alarmes CloudWatch surveillent les métriques de l'infrastructure, le service SNS notifie les fonctions Lambda, qui analysent l'alerte, choisissent la remédiation appropriée (redémarrage de service via Systems Manager, scale-up d'un Auto Scaling Group, retour en arrière d'un déploiement) et l'exécutent. Un verrou anti-flapping stocké dans DynamoDB avec expiration automatique (TTL) empêche l'exécution répétée d'une même remédiation. L'ensemble de l'infrastructure est provisionné de manière industrialisée grâce à Terraform, selon les principes de l'Infrastructure as Code (IaC).

Le présent rapport décrit l'intégralité du cycle de réalisation du projet, de l'analyse des besoins à la mise en production, en passant par la conception conceptuelle, l'implémentation, les tests et les résultats obtenus. Il est composé de trois chapitres :

- **Le premier chapitre** présente le contexte du projet, la problématique, l'étude et la critique des solutions existantes, la solution proposée ainsi que la méthodologie adoptée.
- **Le deuxième chapitre** expose l'étude conceptuelle : identification des acteurs, spécification des besoins fonctionnels et non fonctionnels, modélisation UML (cas d'utilisation, diagrammes de séquence, diagramme de classes) et conception de l'architecture technique.
- **Le troisième chapitre** décrit la réalisation technique : l'environnement logiciel, le déploiement de l'infrastructure avec Terraform, l'implémentation des services AWS, les tests et la validation, ainsi que les difficultés rencontrées et les solutions apportées.

Enfin, une conclusion générale synthétise les apports du projet et ouvre sur des perspectives d'amélioration.

Chapitre

1

CONTEXTE ET ANALYSE DES BESOINS

1.1 INTRODUCTION

Ce chapitre expose les bases du projet en présentant le contexte de l'exploitation cloud, en étudiant les solutions existantes, en dégagant leurs limites et en définissant les besoins spécifiques auxquels répond notre agent DevOps. La méthodologie adoptée pour conduire le développement est également détaillée.

1.2 CONTEXTE DU PROJET

Smartovate LTD exploite des applications déployées sur le cloud Amazon Web Services. Comme toute infrastructure de production, celle-ci est soumise à des incidents récurrents : hausse anormale de l'utilisation CPU, interruption d'un service applicatif (nginx), ou déploiement défectueux. Traditionnellement, la gestion de ces incidents repose sur la surveillance par des tableaux de bord et l'intervention manuelle des ingénieurs DevOps, qui diagnostiquent le problème puis exécutent la remédiation appropriée. Cette approche présente deux limitations majeures : elle induit un délai de réaction non négligeable (le temps pendant lequel le service reste dégradé), et elle mobilise des ressources humaines sur des tâches répétitives et bien identifiées. L'automatisation de la boucle complète « détection → diagnostic → remédiation » constitue donc un gain à la fois en qualité de service et en productivité des équipes, et c'est précisément l'objectif de ce stage.

Le projet s'inscrit dans la démarche du DevOps et de l'observabilité : l'infrastructure est décrite en code (Terraform), la surveillance est déclarative (CloudWatch), et la remédiation est pilotée par du code serveur (fonctions Lambda).

1.3 PROBLÉMATIQUE

L'exploitation manuelle d'une infrastructure cloud se heurte à plusieurs difficultés. Premièrement, le délai entre l'apparition d'un incident et sa prise en charge humaine peut atteindre plusieurs

minutes, pendant lesquelles le service reste dégradé ou indisponible. Deuxièmement, les incidents de même nature (surcharge CPU, service arrêté) se reproduisent fréquemment et appellent toujours les mêmes actions correctives, ce qui en fait des candidats idéaux à l'automatisation. Troisièmement, l'automatisation naïve comporte un risque : sans garde-fou, un mécanisme de remédiation peut se déclencher en boucle (phénomène de *flapping*) et aggraver la situation, par exemple en multipliant les instances de calcul sans résoudre la cause racine. Il devient donc nécessaire de concevoir une solution capable d'assurer une détection intelligente, un diagnostic fiable, une remédiation appropriée et une protection contre les répétitions excessives, tout en garantissant la traçabilité de chaque action.

1.4 ÉTUDE DE L'EXISTANT

Afin d'identifier les bonnes pratiques et les lacunes des solutions actuelles, nous avons étudié plusieurs approches de la gestion d'incidents. Le tableau 1.1 présente une étude comparative entre les solutions existantes.

TABLE 1.1 – Étude comparative des solutions existantes

Approche / Outil	Principe	Détection	Remédiation auto.	Anti-flapping
Surveillance manuelle (CloudWatch, Grafana)	Tableaux de bord consultés par un opérateur	Oui (métriques)	Non (humaine)	Non applicable
AWS Auto Scaling natif	Ajustement automatique de la capacité	Oui (métriques)	Partielle (scaling)	Oui (stabilisation)
AWS Systems Manager (Run Command)	Exécution de commandes sur un parc	Non	Oui (actions SSM)	Non
Outils AIOps (incident.io, PagerDuty)	Agrégation d'alertes, playbooks	Oui	Assistée	Partielle
Scripts ad hoc (cron, CLI)	Scripts maison périodiques	Limitée	Oui	Rarement
Agent DevOps (proposé)	Chaîne CloudWatch → SNS → Lambda → remédiation + verrou	Oui (alarmes)	Oui (3 actions)	Oui (verrou DynamoDB + TTL)

D'après le tableau 1.1, les solutions existantes couvrent partiellement le besoin : la surveillance détecte bien les incidents mais ne remédie pas, les outils de remédiation exécutent des actions mais sans diagnostic intelligent, et aucune solution intégrée ne combine diagnostic automatique, choix de l'action de remédiation et protection anti-flapping native.

1.5 CRITIQUE DE L'EXISTANT

L'analyse des solutions existantes met en évidence plusieurs limites :

- **Intervention humaine obligatoire** : les outils de surveillance se contentent d'alerter, le diagnostic et l'action restent manuels, ce qui allonge le temps de résolution.
- **Remédiation aveugle** : les outils d'automatisation exécutent des actions prédéfinies sans analyser la nature de l'alerte.
- **Absence de protection anti-flapping** : sans mécanisme de verrouillage, une automatisation répète indéfiniment la même action, ce qui peut aggraver l'incident (sur-provisionnement, boucles de redémarrage).
- **Manque d'industrialisation** : les scripts ad hoc ne sont pas versionnés, traçables ni reproductibles, contrairement aux principes de l'Infrastructure as Code.
- **Traçabilité insuffisante** : peu de solutions conservent un journal centralisé des décisions et actions effectuées (CloudTrail, CloudWatch Logs).

1.6 SOLUTION PROPOSÉE

L'agent DevOps proposé répond à ces limitations par une chaîne complètement automatisée et sécurisée :

- **Détection** : des alarmes CloudWatch surveillent les métriques critiques (CPU > 80 % pendant 5 minutes, taux d'erreurs) et publient sur un topic SNS.
- **Ingestion** : une fonction Lambda `ingestion` reçoit la notification SNS, la valide et l'enrichit.

- **Diagnostic** : la fonction Lambda remediation analyse l'alarme et la classe dans une catégorie (CPU_OVERLOAD, SERVICE_DOWN, DEPLOYMENT_ERROR) à l'aide d'un modèle d'IA (Amazon Bedrock, avec bascule automatique en mode démo si le quota n'est pas activé).
- **Décision** : selon la catégorie, l'agent choisit l'action parmi trois documents SSM : restart-service, scale-up et rollback-deployment.
- **Protection anti-flapping** : avant toute exécution, l'agent vérifie dans une table DynamoDB qu'aucun verrou n'existe pour la même cible ; si un verrou actif est trouvé, l'action est refusée avec un code HTTP 429. Après exécution, un verrou est créé avec un TTL de 10 minutes.
- **Industrialisation** : l'intégralité des 39 ressources AWS est provisionnée par Terraform, garantissant reproductibilité et traçabilité.

La figure 1.1 présente une carte mentale de l'ensemble des composants de l'agent DevOps et de leurs interactions : surveillance, diagnostic, remédiation, protection, sécurité, infrastructure et Infrastructure as Code.

Figure 1 — Carte Mentale du Projet Agent DevOps



FIGURE 1.1 – Carte mentale des composants de l’agent DevOps

La figure 1.2 présente l’architecture générale de la solution, organisée autour du flux Alarme CloudWatch → SNS → Lambda Ingestion → Lambda Remediation → actions SSM / Auto Scaling Group, avec le verrou anti-flapping dans DynamoDB.

Figure 2 — Architecture Globale du Système

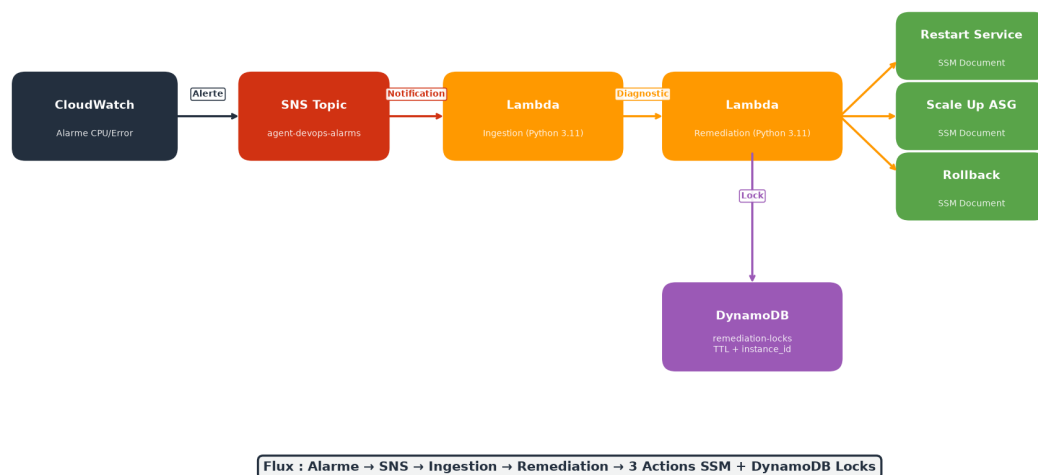


FIGURE 1.2 – Architecture générale de la solution

1.7 MÉTHODOLOGIE ADOPTÉE

Le projet a été conduit selon la méthodologie Scrum, une approche agile itérative reposant sur des cycles courts appelés *sprints*. Les rôles étaient répartis comme suit : le Product Owner (l'encadrant professionnel) définissait la vision et les priorités, et l'équipe de développement (le stagiaire) réalisait les tâches. Les cérémonies (planification, revue de sprint, rétrospective) ont permis d'ajuster le périmètre en fonction des contraintes rencontrées, notamment le quota Amazon Bedrock non activé qui a conduit à la mise en place d'un mode démo automatique.

TABLE 1.2 – Planification des sprints

Sprint	Période	Objectifs	Livrables
Sprint 1	Semaines 1–2	Socle infrastructure (IaC)	VPC, sous-réseaux, EC2, ALB, ASG via Terraform
Sprint 2	Semaines 3–4	Surveillance et notification	Alarmes CloudWatch, topic SNS, CloudTrail, groupes de logs
Sprint 3	Semaines 5–6	Agent intelligent	Lambdas ingestion/remediation, documents SSM, table DynamoDB, anti-flapping
Sprint 4	Semaines 7–8	Tests et documentation	Tests end-to-end, preuves CLI/console, rapport de stage

1.8 CONCLUSION

L'analyse des besoins a permis de dégager un périmètre fonctionnel précis : automatiser la boucle détection-diagnostic-remédiation avec une protection anti-flapping, sur une infrastructure entièrement codée en Terraform. Le chapitre suivant présente la conception conceptuelle de cette solution.

Étude conceptuelle

Sommaire

1.1	INTRODUCTION	2
1.2	CONTEXTE DU PROJET	2
1.3	PROBLÉMATIQUE	2
1.4	ÉTUDE DE L'EXISTANT	3
1.5	CRITIQUE DE L'EXISTANT	4
1.6	SOLUTION PROPOSÉE	4
1.7	MÉTHODOLOGIE ADOPTÉE	7
1.8	CONCLUSION	7

2.1 INTRODUCTION

Ce chapitre présente la modélisation du système : identification des acteurs, spécification des besoins fonctionnels et non fonctionnels, diagrammes UML (cas d'utilisation, séquences, classes) et conception de l'architecture technique (réseau et flux de données).

2.2 IDENTIFICATIONS DES ACTEURS

Le tableau 2.1 présente l'identification des deux acteurs principaux du système.

TABLE 2.1 – Identification des acteurs

Acteur	Description
Administrateur DevOps	Personne responsable de l'infrastructure. Il déploie l'agent via Terraform, supervise les alarmes et les journaux, et intervient manuellement en cas d'incident non couvert par les remédiations automatisées.
Système de surveillance (CloudWatch)	Acteur système qui génère les alarmes à partir des métriques de l'infrastructure (CPU, taux d'erreurs) et déclenche la chaîne de remédiation via SNS.

2.3 IDENTIFICATIONS DES BESOINS

2.3.1 BESOINS FONCTIONNELS

Les besoins fonctionnels ont été listés dans le tableau 2.2 avec des identifiants uniques pour faciliter le suivi.

TABLE 2.2 – Besoins fonctionnels

ID	Fonctionnalité	Description détaillée
F1	Surveiller l'infrastructure	Créer des alarmes CloudWatch sur les métriques critiques (CPU > 80 % pendant 5 min, taux d'erreurs) et les publier sur le topic SNS agent-devops-alarms.
F2	Ingérer les alertes	Recevoir la notification SNS dans la Lambda ingestion, la valider et extraire le nom et la description de l'alarme.
F3	Diagnostiquer l'incident	Classifier l'alarme (CPU_OVERLOAD, SERVICE_DOWN, DEPLOYMENT_ERROR) avec un niveau de confiance, via Amazon Bedrock ou le moteur de démo.
F4	Choisir l'action de remédiation	Sélectionner le document SSM approprié (restart-service, scale-up, rollback-deployment) selon la catégorie diagnostiquée.
F5	Exécuter la remédiation	Lancer l'exécution du document SSM sur les instances cibles ou déclencher le scale-up de l'ASG.
F6	Protéger contre le flapping	Vérifier l'existence d'un verrou dans DynamoDB avant toute exécution ; refuser avec un code 429 si un verrou actif existe ; créer un verrou avec TTL de 10 minutes après exécution.
F7	Tracer l'activité	Journaliser chaque étape (réception, diagnostic, décision, exécution, blocage) dans CloudWatch Logs et CloudTrail.
F8	Provisionner l'infrastructure	Déployer les 39 ressources de manière reproductible via Terraform (IaC).

2.3.2 BESOINS NON FONCTIONNELS

Les besoins non fonctionnels du projet sont présentés dans le tableau 2.3.

TABLE 2.3 – Besoins non fonctionnels

Catégorie	Exigences
Réactivité	Remédiation déclenchée en moins d’une minute après le franchissement du seuil de l’alarme.
Fiabilité	Le mécanisme anti-flapping doit bloquer toute répétition d’action sur une même cible pendant le cooldown (10 minutes).
Sécurité	Moindre privilège : utilisateur IAM dédié (agent-devops-user), MFA sur le compte, rôles IAM distincts pour chaque Lambda.
Traçabilité	Chaque action journalisée (CloudWatch Logs) et auditable (CloudTrail).
Maintenabilité	Infrastructure entièrement codée avec Terraform; code modulaire et documenté.
Coût	Exploitation dans les limites du Free Tier AWS (instances micro, Lambdas à faible volumétrie).

2.4 DIAGRAMME DE CAS D’UTILISATION

La figure 2.1 présente le diagramme de cas d’utilisation global de l’agent DevOps : déploiement de l’infrastructure, réception et diagnostic des alarmes, exécution et blocage des remédiations, et supervision des journaux.

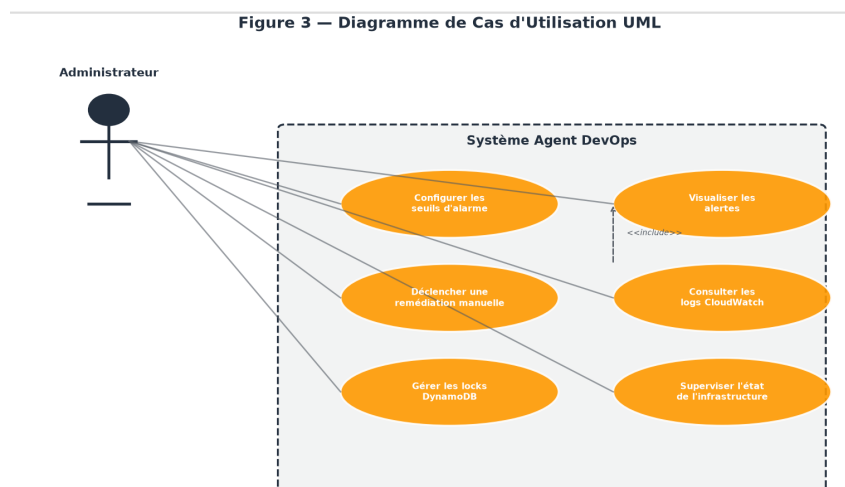


FIGURE 2.1 – Diagramme de cas d’utilisation global

2.5 DIAGRAMMES DE SÉQUENCE

2.5.1 SCÉNARIO CLÉ : DÉTECTION ET RÉMEDIATION D'UN INCIDENT

La figure 2.2 présente le cycle complet de remédiation : CloudWatch publie l'alarme sur SNS (1), SNS invoque la Lambda ingestion (2), celle-ci transmet le diagnostic à la Lambda remediation (3), qui vérifie le verrou dans DynamoDB (4), choisit l'action, exécute le document SSM ou le scale-up (5) et crée le verrou avec TTL (6).

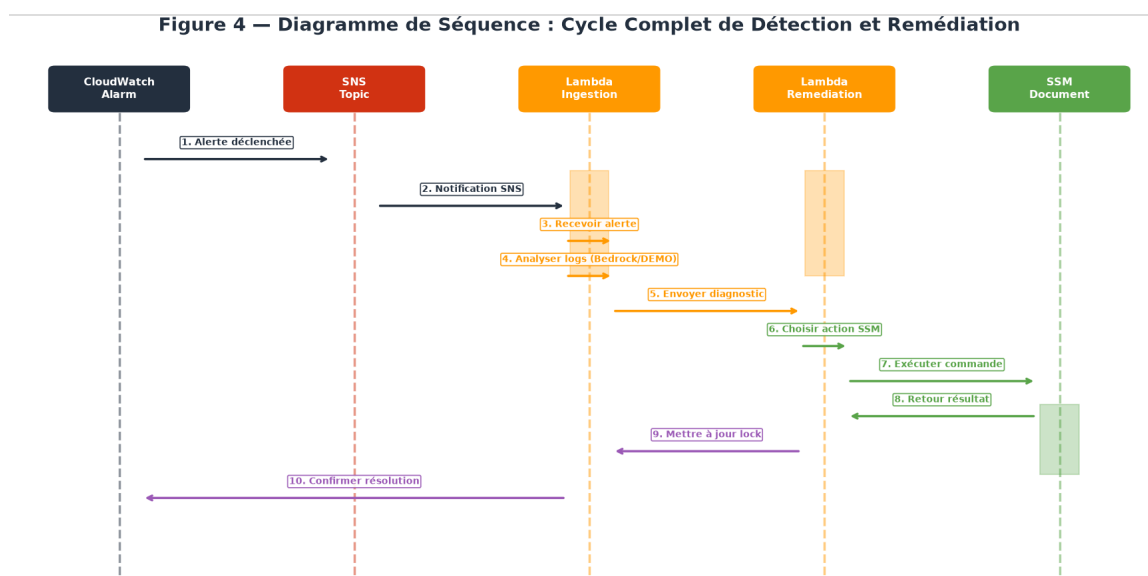


FIGURE 2.2 – Diagramme de séquence – Détection et remédiation d'un incident

2.5.2 SCÉNARIO CLÉ : BLOCAGE ANTI-FLAPPING

La figure 2.3 décrit le scénario de protection : lors d'un second appel de remédiation pour la même cible, la Lambda remediation détecte le verrou actif dans DynamoDB et retourne une réponse HTTP 429 invitant à attendre la fin du cooldown.

Figure 5 — Diagramme de Séquence : Mécanisme Anti-Flapping (Verrou Existant → HTTP 429)

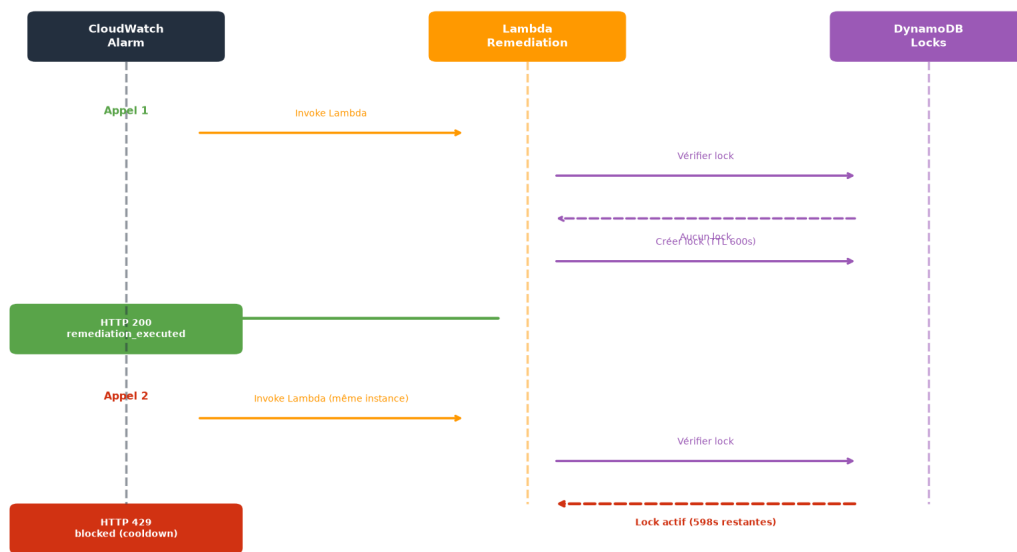


FIGURE 2.3 – Diagramme de séquence – Blocage anti-flapping

2.6 DIAGRAMME DE CLASSES

La figure 2.4 présente le diagramme de classes du système, structuré autour de quatre entités : **Alarme** (nom, description, seuil, état), **Diagnostic** (catégorie, confiance, cause racine), **RemediationAction** (type, cible, statut) et **Lock** (instance_id, action, horodatage, TTL). Une alarme donne lieu à un diagnostic, qui déclenche au plus une action de remédiation, elle-même protégée par un verrou.

Figure 6 — Diagramme de Classes UML

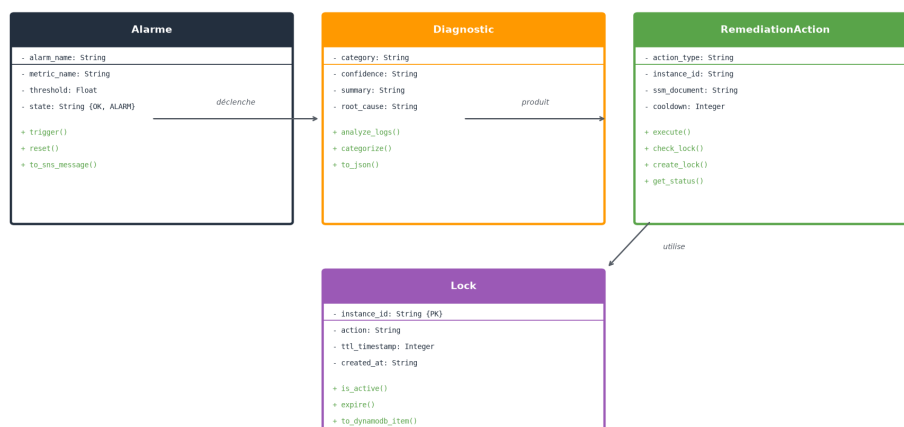


FIGURE 2.4 – Diagramme de classes du système

2.7 CYCLE DE VIE D'UNE ALERTE

La figure 2.5 illustre les états traversés par une alerte CloudWatch : la métrique est d'abord à l'état OK ; lorsque le seuil est franchi (CPU > 80 % pendant 5 minutes), l'alarme passe à l'état ALARM et publie sur SNS. Le diagnostic mène ensuite à une décision : soit une remédiation est exécutée et l'alarme retourne à OK, soit le verrou anti-flapping bloque l'action jusqu'à l'expiration du TTL.

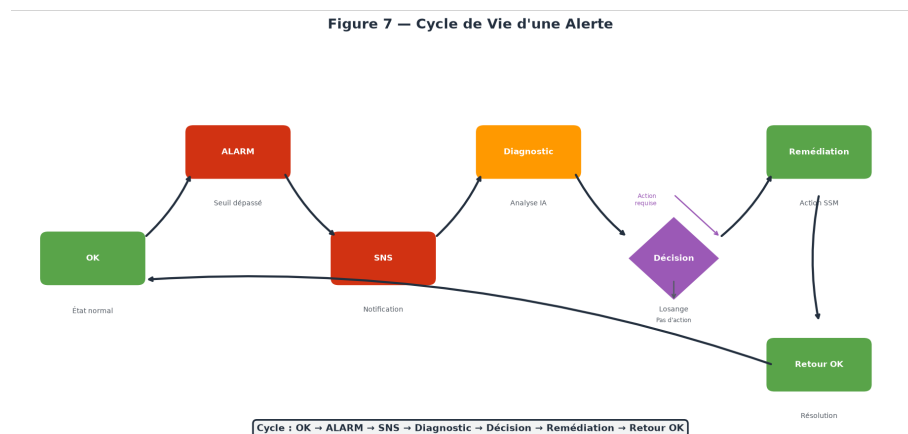


FIGURE 2.5 – Cycle de vie d'une alerte CloudWatch

2.8 CONCEPTION DE L'ARCHITECTURE TECHNIQUE

2.8.1 ARCHITECTURE RÉSEAU

La figure 2.6 présente l'architecture réseau déployée : un VPC agent-devops-vpc en 10.0.0.0/16, comportant deux sous-réseaux publics (public-1 en us-east-1a et public-2 en us-east-1b), une passerelle Internet, une table de routage, un groupe de sécurité, un ALB répartissant le trafic vers les instances EC2 du groupe Auto Scaling, ainsi qu'une base de données relationnelle (RDS).

Figure 8 — Architecture Réseau VPC

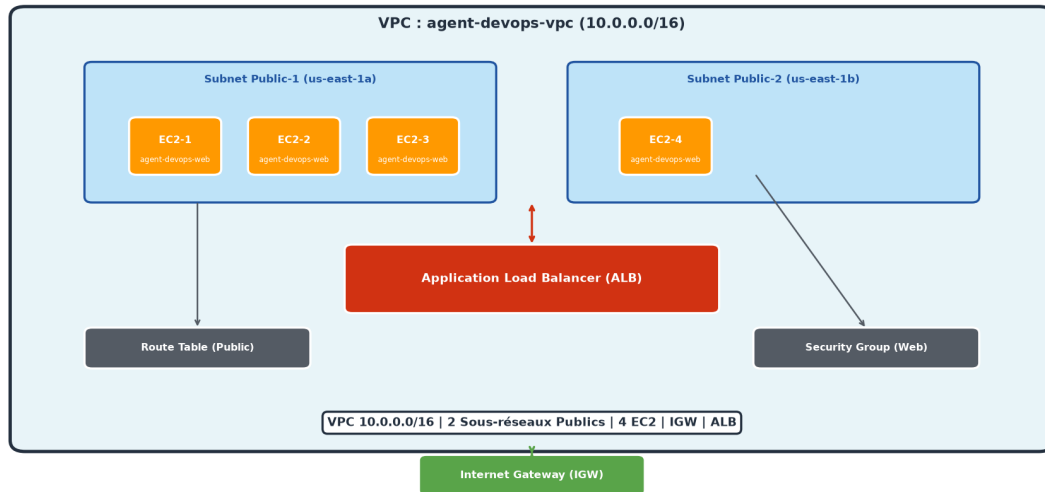


FIGURE 2.6 – Architecture réseau du VPC 10.0.0.0/16

2.8.2 FLUX DE DONNÉES END-TO-END

La figure ?? présente le flux de données complet entre les huit composants de la solution : métriques EC2 → CloudWatch → SNS → Lambda ingestion → Lambda remediation → DynamoDB (verrou) et SSM / ASG, avec journalisation dans CloudWatch Logs et audit dans CloudTrail.

Figure 9 — Flux de Données entre les Composants

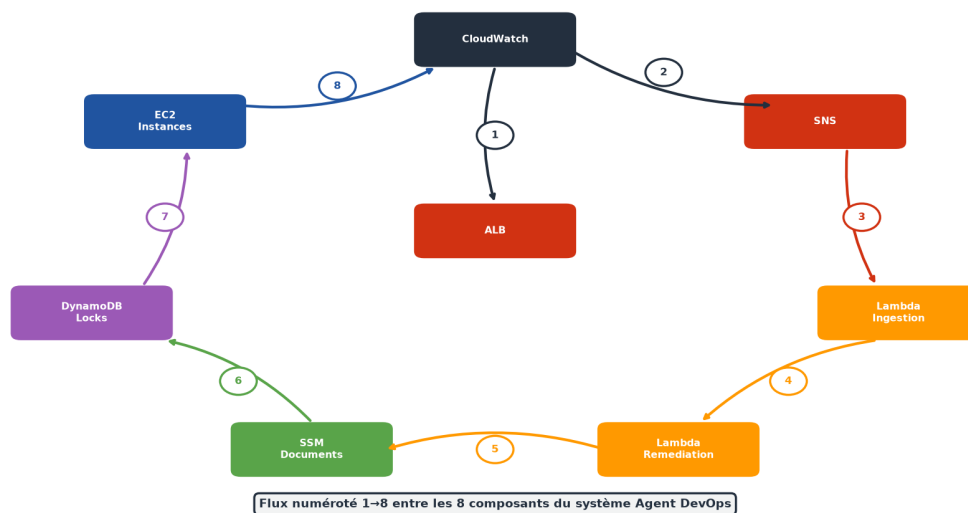


FIGURE 2.7 – Flux de données end-to-end entre les services AWS

2.9 CONCLUSION

La conception conceptuelle a permis de formaliser les besoins et de modéliser les interactions du système. Les choix retenus — chaîne d'événements AWS, verrou anti-flapping dans DynamoDB et provisionnement Terraform — structurent l'implémentation présentée au chapitre suivant.

Réalisation et résultats

Sommaire

2.1	INTRODUCTION	9
2.2	IDENTIFICATIONS DES ACTEURS	9
2.3	IDENTIFICATIONS DES BESOINS	9
2.3.1	BESOINS FONCTIONNELS	9
2.3.2	BESOINS NON FONCTIONNELS	10
2.4	DIAGRAMME DE CAS D'UTILISATION	11
2.5	DIAGRAMMES DE SÉQUENCE	12
2.5.1	SCÉNARIO CLÉ : DÉTECTION ET RÉMEDIATION D'UN INCIDENT	12
2.5.2	SCÉNARIO CLÉ : BLOCAGE ANTI-FLAPPING	12
2.6	DIAGRAMME DE CLASSES	13
2.7	CYCLE DE VIE D'UNE ALERTE	14
2.8	CONCEPTION DE L'ARCHITECTURE TECHNIQUE . . .	14
2.8.1	ARCHITECTURE RÉSEAU	14
2.8.2	FLUX DE DONNÉES END-TO-END	15
2.9	CONCLUSION	16

3.1 INTRODUCTION

Ce chapitre est consacré à la réalisation du projet et à l'évaluation des résultats obtenus. Il présente l'environnement logiciel, la structure du projet, le déploiement de l'infrastructure avec Terraform, l'implémentation des services AWS, puis les tests et la validation de la chaîne complète de remédiation automatique.

3.2 ENVIRONNEMENT LOGICIEL

Le développement et le déploiement du projet ont nécessité les outils suivants :

- **Cloud** : Amazon Web Services (région us-east-1), compte 718897321963.
- **Infrastructure as Code** : Terraform (provider AWS v5.100.0).
- **Langages** : Python 3.11 (fonctions Lambda), HCL (Terraform), scripts CLI.
- **AWS CLI** : authentification via l'utilisateur IAM dédié agent-devops-user (clé d'accès), MFA activé sur le compte root.
- **IDE** : Visual Studio Code. **Versioning** : Git.

TABLE 3.1 – Récapitulatif des outils et versions utilisés

Outil / Service	Version / Réf.	Rôle dans le projet
Terraform	provider AWS v5.100.0	Provisionnement des 39 ressources
Python	3.11	Runtime des fonctions Lambda
EC2 / ASG / ALB	agent-devops-asg, agent-devops-alb-v2	Instances web, Auto Scaling, load balancer
CloudWatch	alarmes agent-devops-cpu-high, agent-devops-error-rate	Alarmes (cpu-high, error-rate) et journaux
SNS	topic agent-devops-alarms	Topic agent-devops-alarms
Lambda	agent-devops-ingestion, agent-devops-remediation	Fonctions ingestion et remédiation
DynamoDB	table agent-devops-remediation-locks	Table remediation-locks avec TTL
Systems Manager	docs restart-service, scale-up, rollback	Documents restart-service, scale-up, rollback
Bedrock	mode DEMO (quota non activé)	Diagnostic IA (mode démo si quota non activé)
CloudTrail / IAM	trail agent-devops-trail, user agent-devops-user	Audit des API / moindre privilège

3.3 IMPLÉMENTATION

3.3.1 STRUCTURE DU PROJET

La figure 3.1 présente l'arborescence du projet, organisée en quatre dossiers principaux : infra/ (Terraform par couche), lambda/ (code Python), scripts/ (tests de validation) et documentation/.



FIGURE 3.1 – Arborescence du projet

3.3.2 DÉPLOIEMENT DE L'INFRASTRUCTURE AVEC TERRAFORM

L'infrastructure a été déployée par couches successives via `terraform apply` : 7 ressources réseau (passerelle Internet, 2 sous-réseaux, groupes de sécurité, tables de routage), l'ALB, 7 ressources de surveillance (groupes de logs, topic SNS, bucket S3, CloudTrail, 2 alarmes), la Lambda ingestion (rôle IAM, politique, fonction, permission SNS, abonnement), puis les documents SSM, la table DynamoDB et la Lambda remediation. La commande `terraform`

state list confirme le déploiement de 39 ressources gérées (les entrées data.aws_ami et data.aws_caller_identity étant des sources de données, non des ressources).

TABLE 3.2 – Ressources AWS déployées par Terraform

Couche	Ressources principales
Réseau	aws_vpc.main, 2 sous-réseaux, passerelle Internet, tables de routage, groupes de sécurité
Compute	Launch template, ASG agent-devops-asg, ALB + listener, 3 instances EC2 agent-devops-web
Surveillance	2 alarmes CloudWatch, topic SNS, bucket S3, CloudTrail, groupes de logs
Serverless	Lambdas ingestion et remediation, rôles/politiques IAM, permission SNS, abonnement
Remédiation	3 documents SSM (restart-service, scale-up, rollback-deployment), rôle SSM
Protection	Table DynamoDB remediation-locks (clé instance_id, TTL sur ttl_timestamp)

3.3.3 SURVEILLANCE : CloudWATCH ET SNS

Deux alarmes CloudWatch ont été configurées : agent-devops-cpu-high (CPU > 80 % pendant 5 minutes) et agent-devops-error-rate. La figure 3.2 montre l’alarme CPU en état ALARM ; la figure 3.3 présente le topic SNS agent-devops-alarms (type Standard) avec son ARN, qui relie la surveillance aux fonctions Lambda.

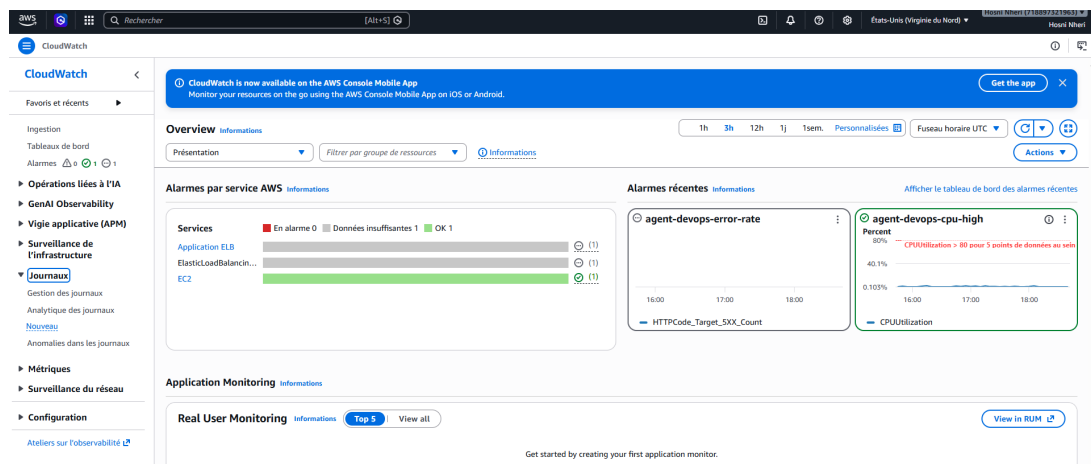


FIGURE 3.2 – Console CloudWatch – alarme CPU élevé en état ALARM

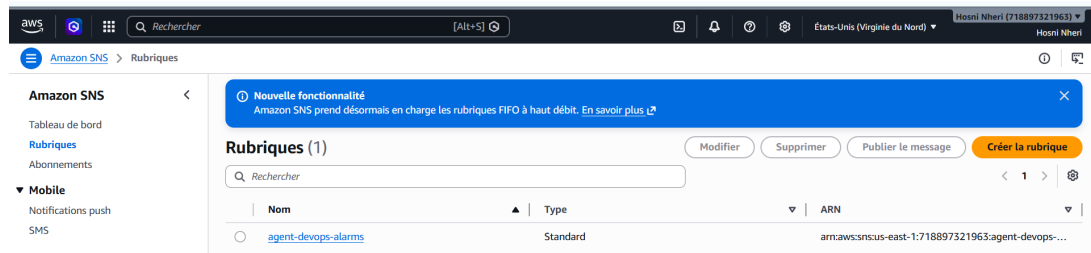


FIGURE 3.3 – Rubrique SNS agent-devops-alarms (type Standard, ARN us-east-1)

3.3.4 FONCTION LAMBDA INGESTION

La fonction Lambda `agent-devops-ingestion` (Python 3.11) est déclenchée par le topic SNS, comme le montre la figure 3.4. Les logs CloudWatch de la figure 3.5 prouvent la réception **réelle** de la notification SNS contenant l’alarme « `agent-devops-cpu-high` » (vrai `MessageId` et souscription effective), démontrant l’orchestration CloudWatch → SNS → Lambda de bout en bout.

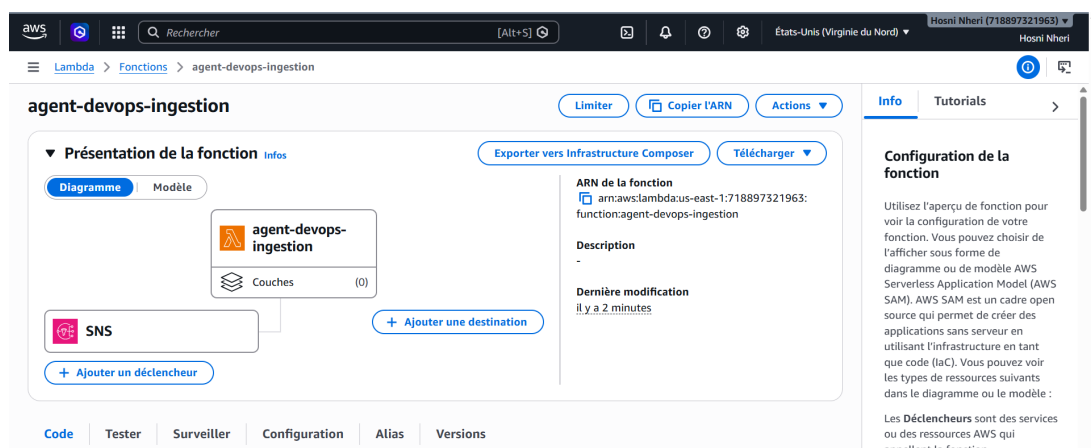


FIGURE 3.4 – Fonction Lambda Ingestion avec son déclencheur SNS et son ARN

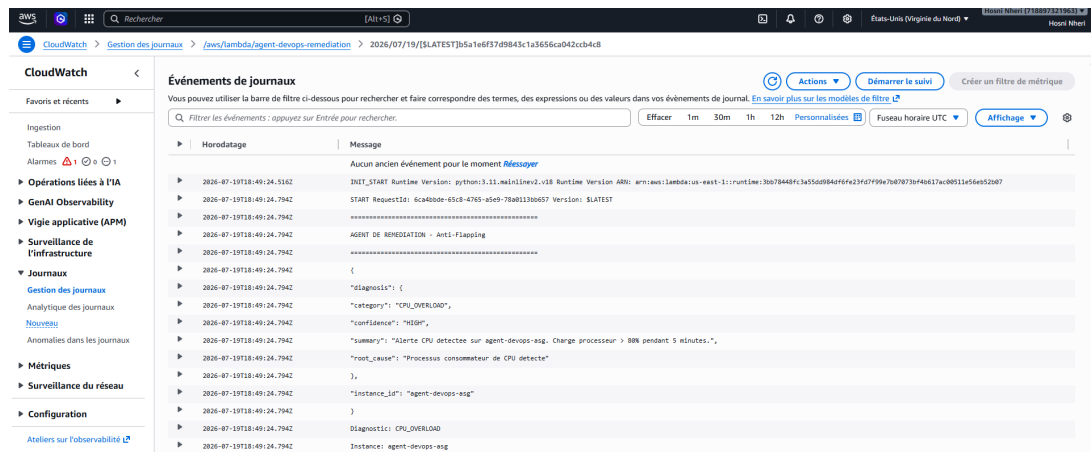


FIGURE 3.5 – Logs CloudWatch – réception réelle de l’alarme SNS « agent-devops-cpu-high »

3.3.5 FONCTION LAMBDA REMÉDIATION ET DIAGNOSTIC

La fonction Lambda `agent-devops-remediation` (figure 3.6) reçoit l’alerte, la diagnostique et la classe dans une catégorie d’incident (`CPU_OVERLOAD`, `SERVICE_DOWN`, `DEPLOYMENT_ERROR`) avec un niveau de confiance. Le diagnostic s’appuie sur Amazon Bedrock avec des exemples *few-shot*; lorsque le quota Bedrock n’est pas activé (erreur `ResourceNotFoundException`), la fonction bascule automatiquement en mode démo et produit le diagnostic localement, garantissant la continuité du service.

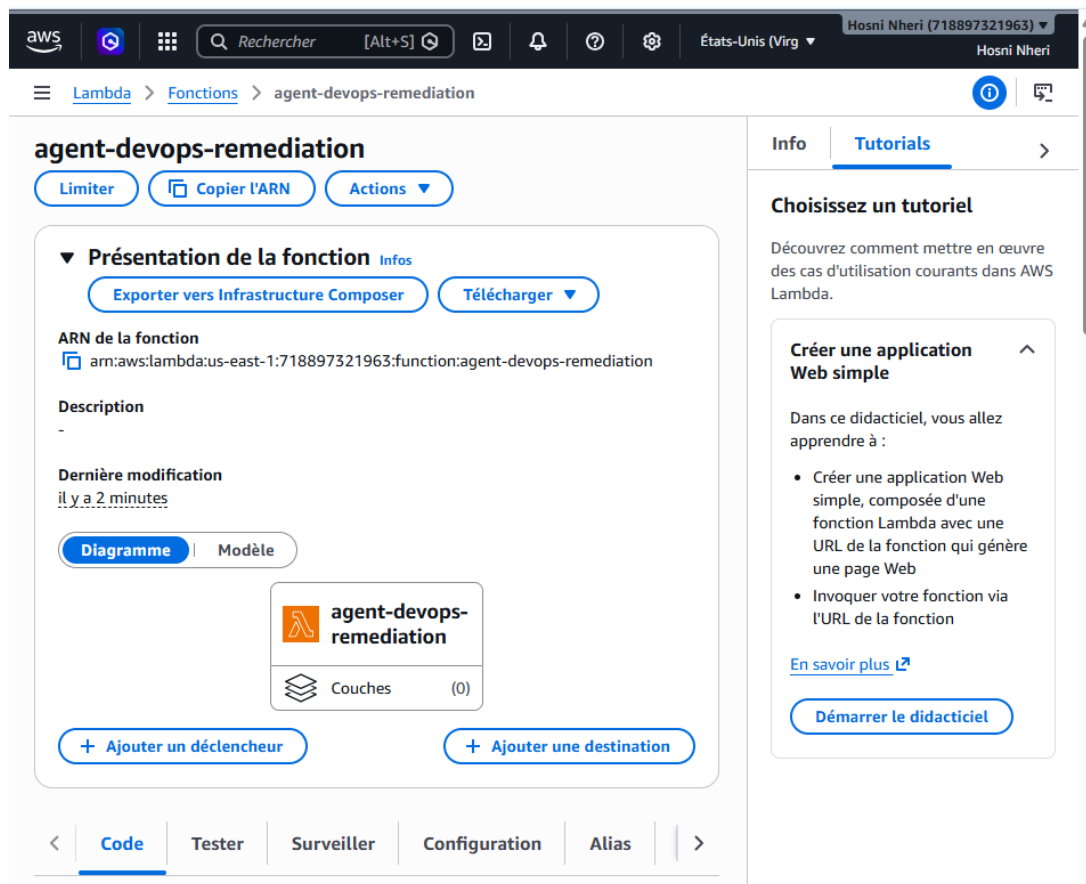


FIGURE 3.6 – Fonction Lambda Remediation présentant son code

3.3.6 VERROU ANTI-FLAPPING AVEC DynamoDB

La protection anti-flapping repose sur la table DynamoDB `remediation-locks` (figure 3.7), dont la clé de partition est `instance_id` et dont l'expiration automatique (TTL) est activée sur `ttl_timestamp`. La figure 3.8 montre un verrou réel créé par la Lambda lors d'une remédiation `scale_up_asg` : cible, action exécutée, date de création et horodatage d'expiration (cooldown de 10 minutes).

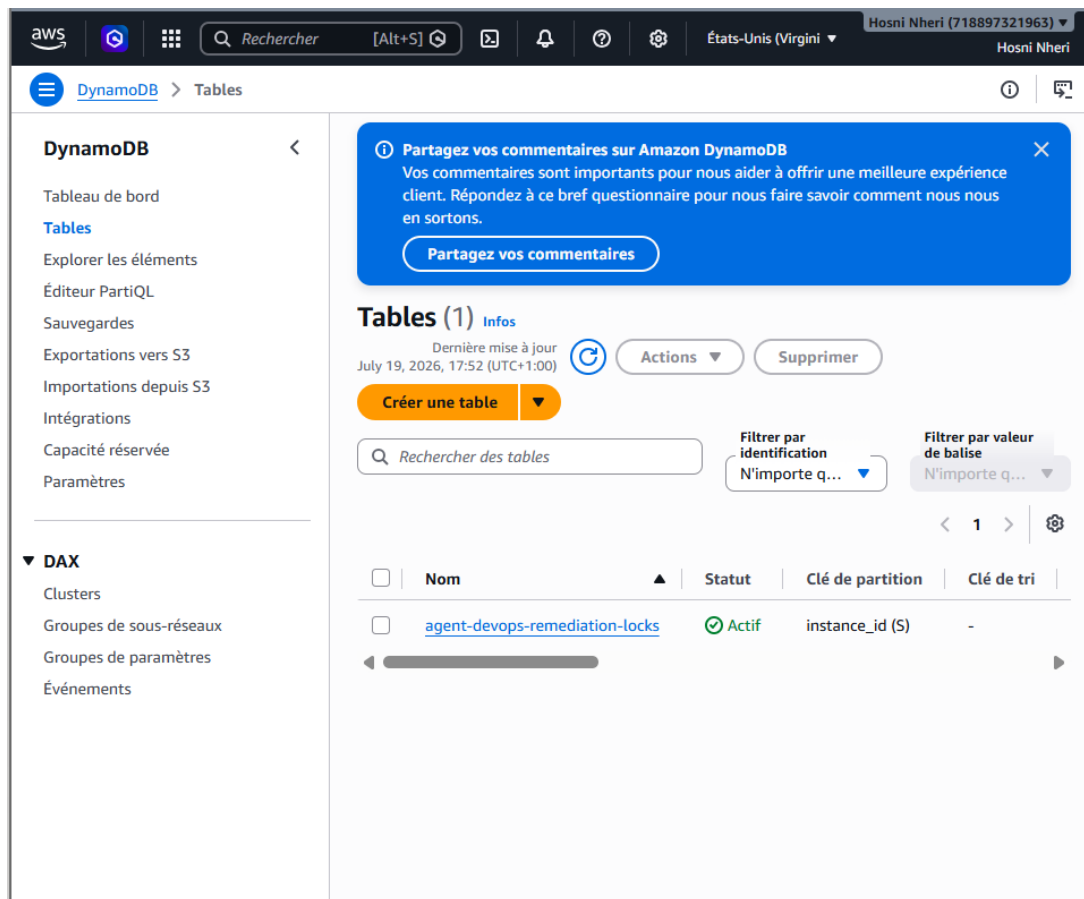


FIGURE 3.7 – Table DynamoDB remediation-locks (Active, TTL activé)

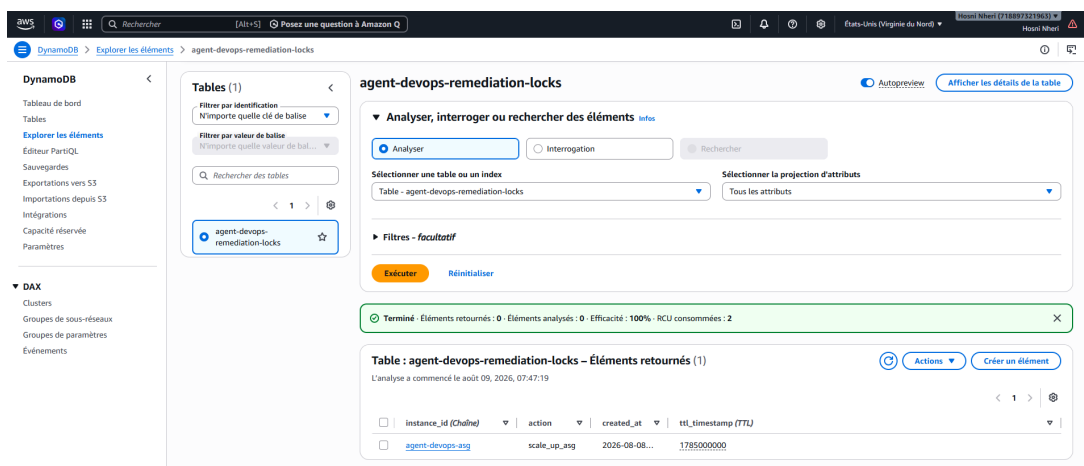


FIGURE 3.8 – Exemple de verrou anti-flapping créé par la Lambda Remediation

3.3.7 REMÉDIATION : DOCUMENTS ET COMMANDES SSM

Trois documents SSM encapsulent les actions de remédiation. Les figures 3.9 et 3.10 montrent les documents restart-service et scale-up dans la console AWS, et le troisième document

agent-devops-rollback via la CLI (statut Active, contenu : `git log --oneline`, `git checkout`, redémarrage du service). La figure 3.11 présente une commande SSM exécutée avec succès sur 2 instances cibles, et la figure 3.12 confirme le redémarrage de nginx (Active: active (running)).

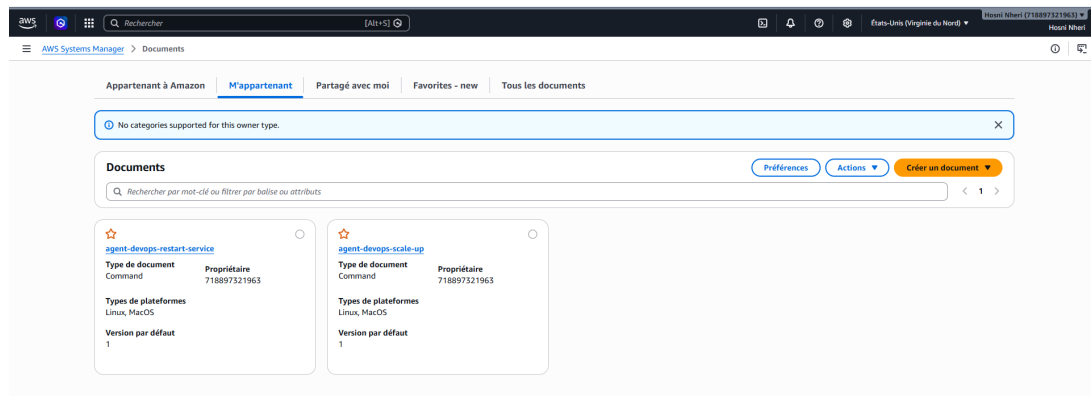


FIGURE 3.9 – Documents SSM (restart-service et scale-up)

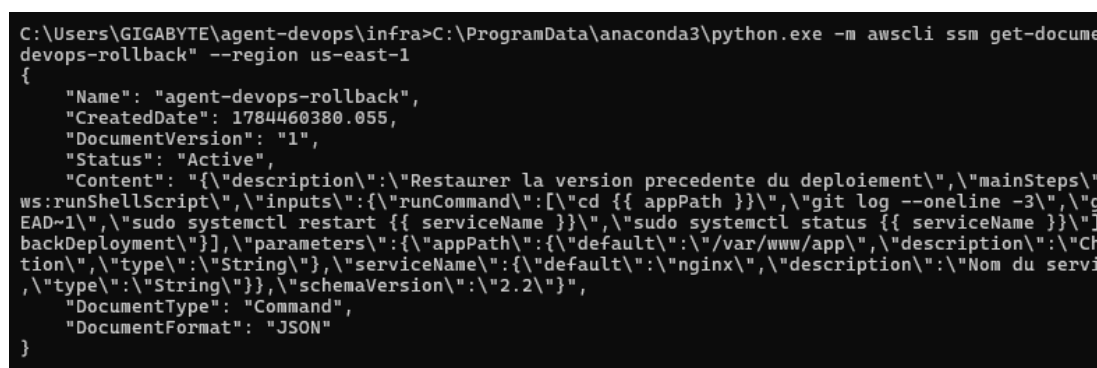


FIGURE 3.10 – Document SSM agent-devops-rollback via CLI (statut Active)

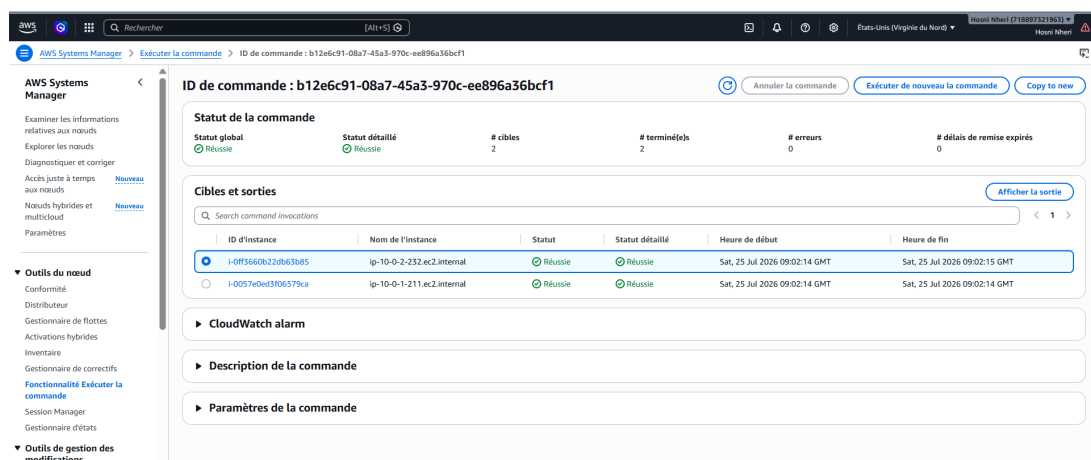


FIGURE 3.11 – Commande SSM exécutée avec succès sur 2 instances cibles

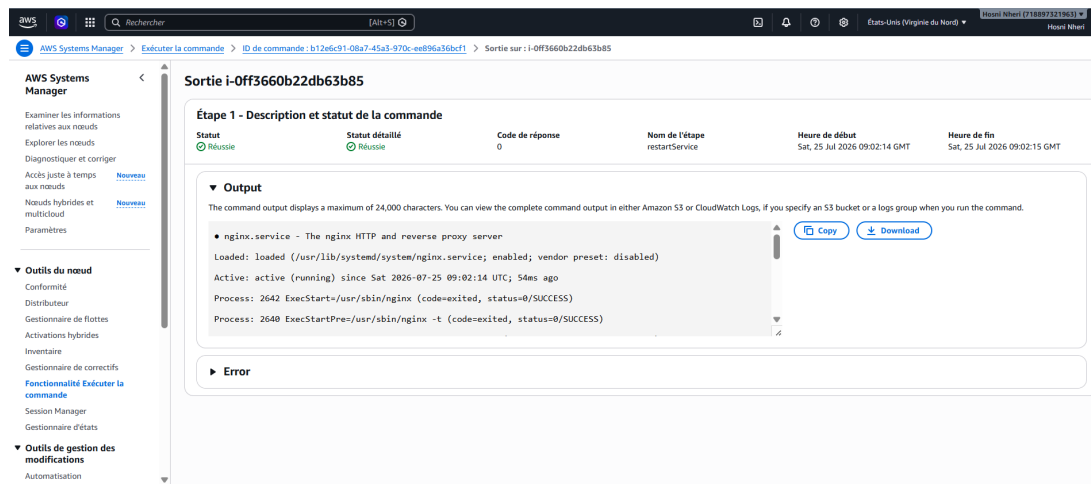


FIGURE 3.12 – Sortie SSM confirmant nginx Active : active (running)

3.3.8 TESTS END-TO-END ET VALIDATION

La validation a été menée en deux temps. Premièrement, la remédiation scale-up : après le déclenchement de l'alarme, le Desired Capacity du groupe Auto Scaling est passé de 1 à 3 (figure 3.14), confirmé par la CLI `aws autoscaling describe-auto-scaling-groups`. Deuxièmement, le mécanisme anti-flapping : une double invocation de la Lambda a produit une première exécution (`remediation_executed`) puis un blocage avec le message « Attendez 10 minutes » et un code HTTP 429 (figure 3.15). La sortie `terraform state list` (figure 3.13) atteste des 39 ressources gérées.

```
C:\Users\GIGABYTE\agent-devops\infra>C:\terraform\terraform.exe state list
data.aws_ami.amazon_linux
data.aws_caller_identity.current
aws_autoscaling_group.web
aws_cloudtrail.main
aws_cloudwatch_log_group.cloudtrail
aws_cloudwatch_metric_alarm.cpu_high
aws_cloudwatch_metric_alarm.error_rate
aws_db_instance.main
aws_db_subnet_group.main
aws_dynamodb_table.remediation_locks
aws_iam_instance_profile.ssm
aws_iam_role.lambda_ingestion
aws_iam_role.lambda_remediation
aws_iam_role.ssm
aws_iam_role.ssm_ec2
aws_iam_role_policy.lambda_ingestion
aws_iam_role_policy.lambda_remediation
aws_iam_role_policy.ssm
aws_iam_role_policy.ssm_ec2
aws_internet_gateway.main
aws_lambda_function.ingestion
aws_lambda_function.remediation
aws_lambda_permission.sns
aws_launch_template.web
aws_lb.web
aws_lb_listener.web
aws_lb_target_group.web
aws_route_table.public
aws_route_table_association.public_1
aws_route_table_association.public_2
aws_s3_bucket.cloudtrail
aws_s3_bucket_policy.cloudtrail
aws_security_group.web
aws_sns_topic.alarms
aws_sns_topic_subscription.lambda
aws_ssm_document.restart_service
aws_ssm_document.rollback_deployment
aws_ssm_document.scale_up_asg
aws_subnet.public_1
aws_subnet.public_2
aws_vpc.main
```

FIGURE 3.13 – Sortie de terraform state list – 39 ressources gérées

```
C:\Users\GIGABYTE\agent-devops\infra>C:\ProgramData\anaconda3\python.exe -m awscli
autoscaling describe-auto-scaling-groups --auto-scaling-group-names agent-devops-as
g --query "AutoScalingGroups[0].DesiredCapacity"
3
```

FIGURE 3.14 – ASG avec Desired Capacity = 3 après remédiation automatique

```
C:\Users\GIGABYTE\agent-devops\infra>type response_test2.json
{"statusCode": 429, "body": "{\"status\": \"blocked\", \"reason\": \"Cooldown anti-flapping actif\", \"message\": \"Une remediation a deja ete effectuee recemment. Attendez 10 minutes.\", \"instance_id\": \"i-test-1234567890\"}"}
}
C:\Users\GIGABYTE\agent-devops\infra>C:\ProgramData\anaconda3\python.exe -m awscli lambda invoke --function-name a
```

FIGURE 3.15 – Réponse HTTP 429 – mécanisme anti-flapping

3.4 TESTS ET VALIDATION

TABLE 3.3 – Tests fonctionnels

Test	Description	Résultat
Réception d'alarme	Alarme cpu-high publiée sur SNS et reçue par la Lambda ingestion (MessageId réel)	OK
Diagnostic	Classification CPU_OVERLOAD avec confiance HIGH (logs Lambda)	OK
Remédiation scale-up	Desired Capacity de l'ASG passé de 1 à 3	OK
Commande SSM	restart-service exécuté sur 2 instances, nginx actif	OK
Anti-flapping	Seconde invocation bloquée (HTTP 429, cooldown ~10 min)	OK
Infrastructure as code	<code>terraform state list</code> : 39 ressources; apply reproductible	OK
Traçabilité	Journaux CloudWatch complets + CloudTrail actif	OK

3.5 DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

TABLE 3.4 – Difficultés rencontrées et solutions

Difficulté	Solution
Quota Amazon Bedrock non activé (ResourceNotFoundException)	Bascule automatique de la Lambda en mode démo : diagnostic généré localement avec les exemples few-shot
Création d'items DynamoDB via CLI (guillemets Windows)	Création de l'item via la console DynamoDB
Instances EC2 non visibles comme « managed » dans SSM Fleet Manager	Utilisation d'une commande SSM historique (25/07/2026) ayant tourné avec succès sur 2 instances comme preuve
Risque de flapping de la remédiation	Verrou DynamoDB avec TTL de 10 minutes et réponse 429
Gestion d'un grand nombre de captures de preuves	Guide de nommage et sélection des preuves les plus significatives (console + CLI)

3.6 CONCLUSION

La réalisation a abouti à un agent DevOps fonctionnel, déployé de manière industrialisée et validé de bout en bout : de la détection par CloudWatch jusqu'à la remédiation automatique et à la protection anti-flapping. Les tests confirment la fiabilité de la chaîne et la valeur ajoutée de l'infrastructure as code.

- [1] Documentation Amazon Web Services. *AWS Documentation*. Disponible sur : <https://docs.aws.amazon.com>
- [2] Terraform – HashiCorp. *Terraform AWS Provider Documentation*. Disponible sur : <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- [3] Documentation AWS Lambda. *Building Lambda functions with Python*. Disponible sur : <https://docs.aws.amazon.com/lambda/latest/dg/lambda-python.html>

- [4] Documentation Amazon DynamoDB. *DynamoDB Time to Live (TTL)*. Disponible sur : <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html>
- [5] Documentation AWS Systems Manager. *Run Command*. Disponible sur : <https://docs.aws.amazon.com/systems-manager/latest/userguide/execute-remote-commands.html>
- [6] Documentation Amazon Bedrock. *User Guide*. Disponible sur : <https://docs.aws.amazon.com/bedrock/latest/userguide>
- [7] AWS CLI – Command Line Interface. Disponible sur : <https://awscli.amazonaws.com/v2/documentation/api/latest/index.html>
- [8] Beyer, B., Jones, C., Petoff, J. et Murphy, N. R. *Site Reliability Engineering : How Google Runs Production Systems*. O'Reilly Media, 2016.
- [9] Kim, G. et al. *The DevOps Handbook*. IT Revolution, 2016.
- [10] Schwaber, K. & Sutherland, J. *The Scrum Guide*. Disponible sur : <https://scrumguides.org>

ANNEXES

Détails de l'utilisateur

Nom d'utilisateur

agent-devops-user

Le nom d'utilisateur peut comporter jusqu'à 64 caractères.
Caractères valides : A-Z, a-z, 0-9 et +, =, @, _ - (tiret)

☐ Fournir aux utilisateurs l'accès à la console de gestion
AWS - facultatif

Outre l'accès à la console, les utilisateurs disposant des autorisations `SignInLocalDevelopmentAccess` peuvent utiliser les mêmes informations d'identification de console pour l'accès par programmation sans avoir besoin de clés d'accès.

i En cas de création d'un accès par programmation à AWS CodeCommit ou Amazon Keyspaces via des clés d'accès ou des informations d'identification spécifiques à un service, vous pourrez les générer après avoir créé cet utilisateur IAM. [En savoir plus](#)

Annuler

Suivant

FIGURE 3.16 – Console IAM – création de l'utilisateur dédié au projet (A.1)

The screenshot shows the AWS IAM console interface. The top navigation bar includes the AWS logo, a search bar, and the user profile 'Hosni Nheri (718897321963)'. The left sidebar shows the 'IAM' menu with options like 'Tableau de bord', 'Gestion des accès', and 'Rapports d'accès'. The main content area displays the details for the user 'agent-devops-user'.

agent-devops-user Infos [Supprimer](#)

Récapitulatif

ARN
arn:aws:iam::718897321963:user/agent-devops-user

Accès à la console
Désactivé

Clé d'accès 1
AKIA[redacted] Active
Jamais utilisé. Créé aujourd'hui.

Création
July 13, 2026, 20:02 (UTC+01:00)

Dernière connexion à la console
-

Clé d'accès 2
[Créer une clé d'accès](#)

FIGURE 3.17 – Utilisateur IAM agent-devops-user – clé d'accès générée (A.2)

3.7 VUE GLOBALE ET RÉSEAU

The screenshot shows the AWS Global View console. The top navigation bar includes the AWS logo, a search bar, and the user profile 'Hosni Nheri (718897321963)'. The left sidebar shows the 'Vue globale AWS' menu with options like 'Explorateur de régions', 'Recherche globale', 'Régions et zones', and 'Paramètres'. The main content area displays a summary of resources deployed across regions.

Récapitulatif
Récapitulatif de vos ressources dans toutes les régions pour lesquelles votre compte est activé.
Récupération de ressources pour toutes les régions sélectionnées
Mise à jour des ressources terminée.
Les totaux des ressources seront inactifs jusqu'à la fin.

Calcul	Stockage	Mise en réseau	Sécurité
13	6	391	37

▼ Afficher le résumé de toutes les ressources

Régions actives	Instances	VPC	Sous-réseaux
17 régions	12 dans 1 région	18 dans 17 régions	57 dans 17 régions
Groupes de sécurité	Volumes	Groupes Auto Scaling	Tables de routage
19 dans 17 régions	3 dans 1 région	1 dans 1 région	18 dans 16 régions
Points de terminaison d'un VPC	Passerelles NAT	Passerelles Internet de sortie uniquement	Passerelles Internet
0 dans 0 régions	0 dans 0 régions	0 dans 0 régions	18 dans 17 régions
Jeux d'options DHCP	IP Elastic	Services de points de terminaison	Listes des préfixes gérés
17 dans 17 régions	5 dans 1 région	0 dans 0 régions	250 dans 17 régions
ACL réseau	Interfaces réseau	Connexions d'appariement de VPC	Réseaux de capacité
18 dans 17 régions	8 dans 1 région	0 dans 0 régions	0 dans 0 régions
Compartiments S3	Clusters RDS	Instances de base de données RDS	Outputs
1 dans 1 région	0 dans 0 régions	2 dans 1 région	0 dans 0 régions
Clusters ECS	Services ECS		
0 dans 0 régions	0 dans 0 régions		

Pour découvrir plus de ressources, rendez-vous sur [Explorateur de ressources AWS](#)

FIGURE 3.18 – AWS Global View – récapitulatif des ressources déployées (A.3)

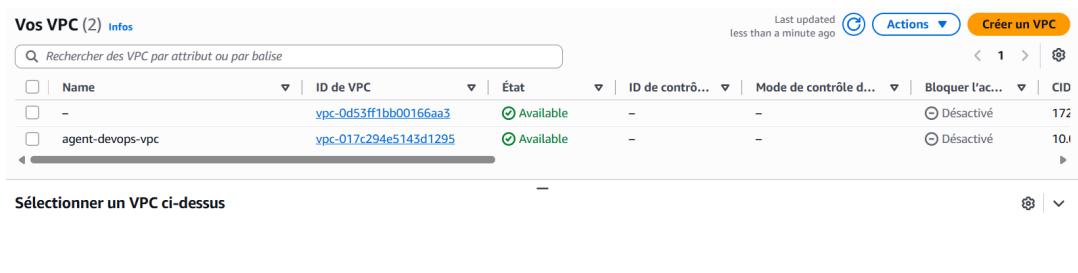


FIGURE 3.19 – Console VPC – agent-devops-vpc disponible (A.4)

3.8 INSTANCES EC2

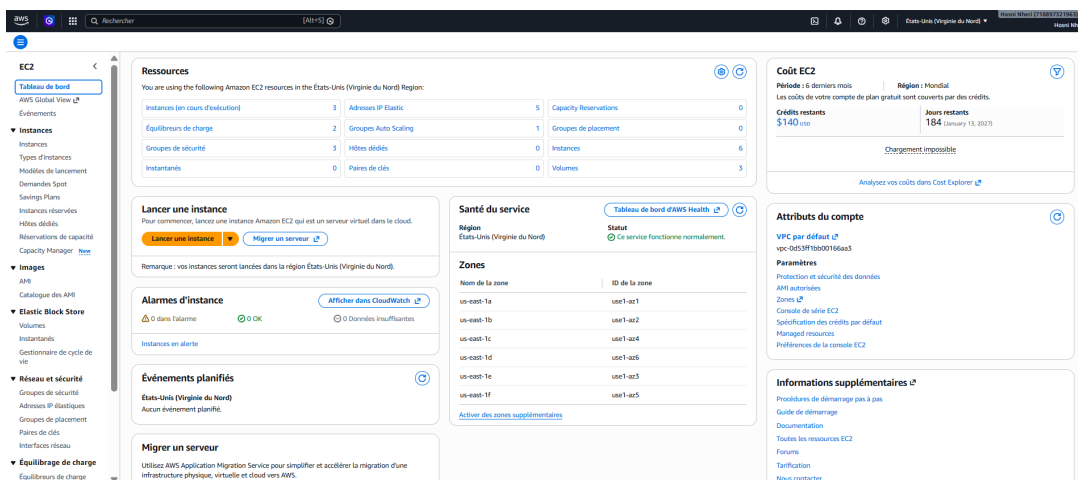


FIGURE 3.20 – Console EC2 – 3 instances web en état running (A.5)

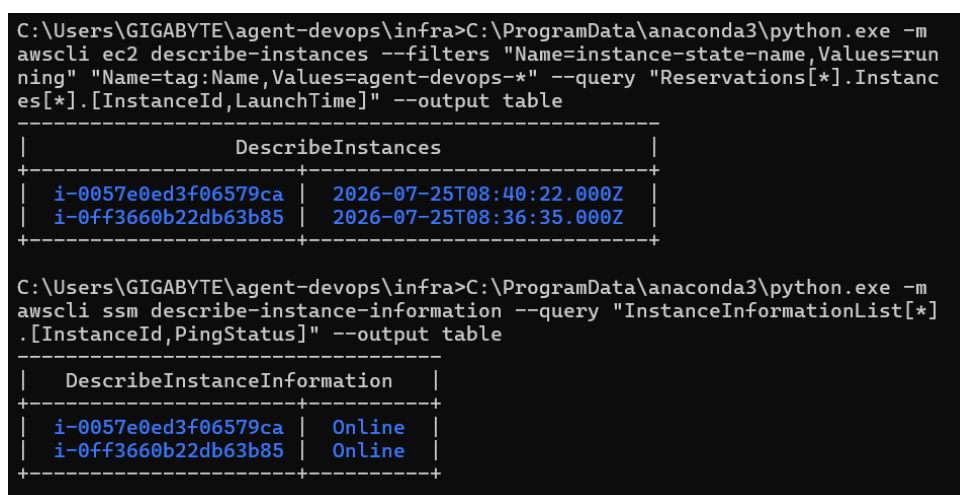


FIGURE 3.21 – CLI – instances EC2 visibles avec statut SSM Online (A.6)

3.9 TESTS DES FONCTIONS LAMBDA

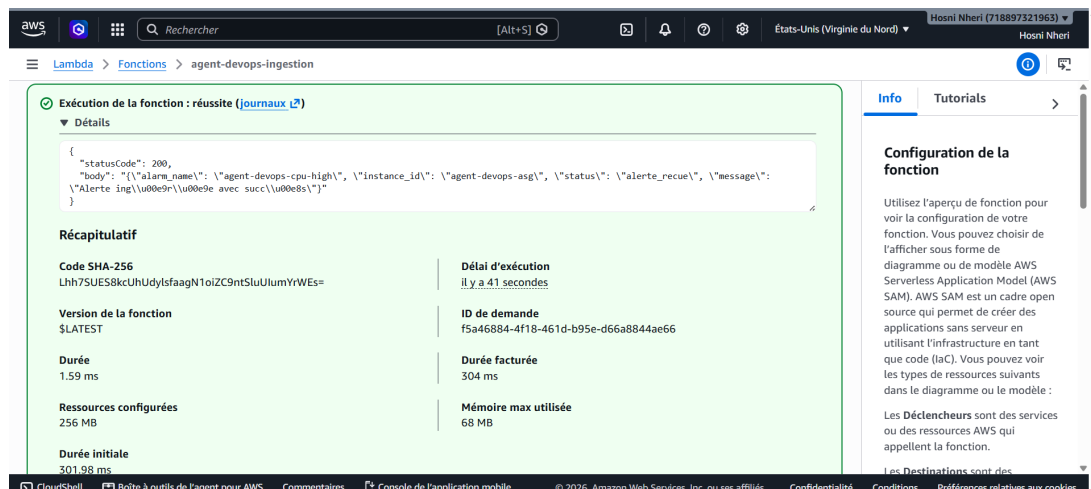


FIGURE 3.22 – Test de la Lambda ingestion – alerte reçue (200) (A.7)

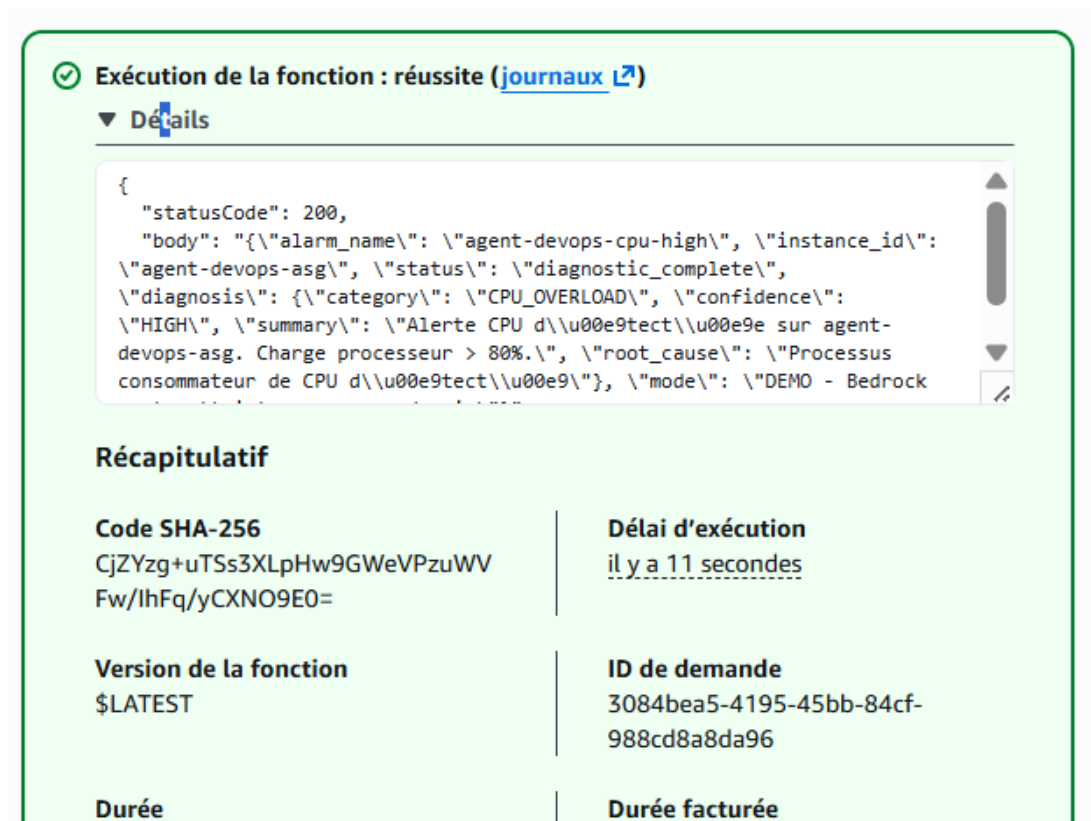


FIGURE 3.23 – Diagnostic automatique – CPU_OVERLOAD, confiance HIGH (A.8)

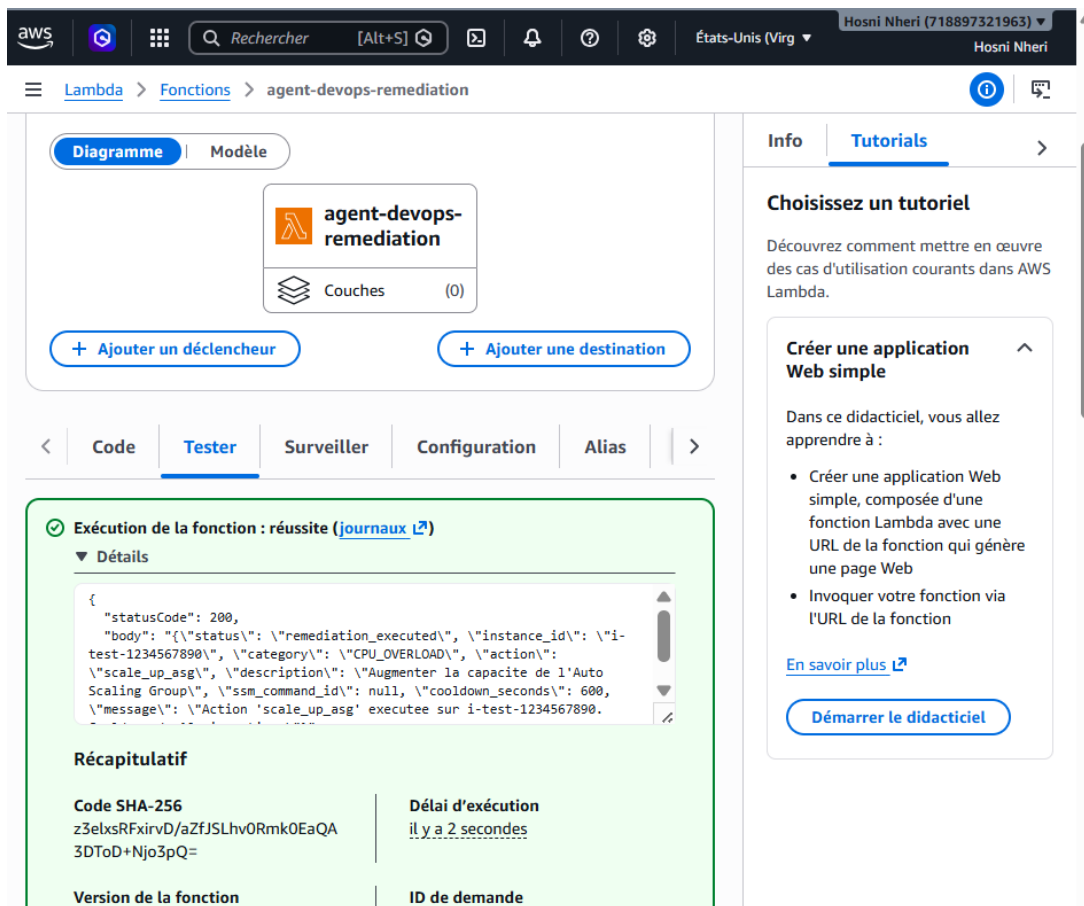


FIGURE 3.24 – Test de la Lambda remediation – action scale_up_asg (A.9)

3.10 DÉPLOIEMENTS TERRAFORM

```
Plan: 7 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_internet_gateway.main: Creating...
aws_subnet.public_2: Creating...
aws_subnet.public_1: Creating...
aws_security_group.web: Creating...
aws_internet_gateway.main: Creation complete after 2s [id=igw-01416cb1282b9f984]
aws_route_table.public: Creating...
aws_route_table.public: Creation complete after 2s [id=rtb-09706acbb6d373115]
aws_security_group.web: Creation complete after 5s [id=sg-048e03b4b5ce95d06]
aws_subnet.public_1: Still creating... [00m10s elapsed]
aws_subnet.public_2: Still creating... [00m10s elapsed]
aws_subnet.public_1: Creation complete after 12s [id=subnet-0c97556e0c0ad95af]
aws_route_table_association.public_1: Creating...
aws_route_table_association.public_1: Creation complete after 1s [id=rtbassoc-0ff30b79c3251af53]
aws_subnet.public_2: Creation complete after 15s [id=subnet-00d1ab25785f64d83]
aws_route_table_association.public_2: Creating...
aws_route_table_association.public_2: Creation complete after 1s [id=rtbassoc-028ae5571039a1373]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

C:\Users\GIGABYTE\agent-devops\infra>
```

FIGURE 3.25 – Terraform apply – déploiement des 7 ressources réseau (A.10)

```

aws_iam_role.lambda_ingestion: Creating...
aws_iam_role.lambda_ingestion: Creation complete after 1s [id=agent-devops-lambda-ingestion-role]
aws_iam_role_policy.lambda_ingestion: Creating...
aws_lambda_function.ingestion: Creating...
aws_iam_role_policy.lambda_ingestion: Creation complete after 1s [id=agent-devops-lambda-ingestion-role:agent-devops-lambda-ingestion-policy]
aws_lambda_function.ingestion: Still creating... [00m10s elapsed]
aws_lambda_function.ingestion: Creation complete after 16s [id=agent-devops-ingestion]
aws_lambda_permission.sns: Creating...
aws_sns_topic_subscription.lambda: Creating...
aws_lambda_permission.sns: Creation complete after 0s [id=AllowSNSInvoke]
aws_sns_topic_subscription.lambda: Creation complete after 1s [id=arn:aws:sns:us-east-1:718897321963:agent-devops-alarms:8f447b5a-fce9-4ce5-87a2-68de56df0d0d]

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

C:\Users\GIGABYTE\agent-devops\infra>

```

FIGURE 3.26 – Terraform apply – déploiement de la Lambda ingestion (A.11)

```

Enter a value: yes

aws_ssm_document.restart_service: Modifying... [id=agent-devops-restart-service]
aws_dynamodb_table.remediation_locks: Creating...
aws_ssm_document.scale_up_asg: Modifying... [id=agent-devops-scale-up]
aws_launch_template.web: Modifying... [id=lt-038d053cb8528db2e]
aws_ssm_document.scale_up_asg: Modifications complete after 2s [id=agent-devops-scale-up]
aws_ssm_document.restart_service: Modifications complete after 2s [id=agent-devops-restart-service]
aws_launch_template.web: Modifications complete after 7s [id=lt-038d053cb8528db2e]
aws_dynamodb_table.remediation_locks: Still creating... [00m10s elapsed]
aws_dynamodb_table.remediation_locks: Creation complete after 16s [id=agent-devops-remediation-locks]

Apply complete! Resources: 1 added, 3 changed, 0 destroyed.

```

FIGURE 3.27 – Terraform apply – déploiement de la table DynamoDB (A.12)

Agent DevOps Autonome pour Infrastructure Auto- Réparatrice

HOSNI NHERI

Résumé :

Ce stage, effectué au sein de Smartovate LTD, a consisté à concevoir et à déployer un agent DevOps autonome capable de détecter et de remédier automatiquement les incidents d'une infrastructure Amazon Web Services. La solution s'appuie sur une chaîne d'événements CloudWatch → SNS → Lambda : l'alarme est diagnostiquée par une fonction Lambda (Amazon Bedrock avec mode démo de secours), classée dans une catégorie d'incident, puis une action de remédiation appropriée est exécutée via AWS Systems Manager (redémarrage de service, scale-up du groupe Auto Scaling, retour en arrière de déploiement). Un verrou anti-flapping stocké dans DynamoDB avec expiration automatique (TTL) protège le système contre les exécutions répétées, tandis que l'intégralité des 39 ressources est provisionnée par Terraform selon les principes de l'Infrastructure as Code. Les tests end-to-end ont validé la remédiation scale-up réelle (passage à 3 instances) et le blocage anti-flapping (HTTP 429).

Mots clés : DevOps, AWS, Serverless, Terraform, Infrastructure as Code, Anti-flapping, Remédiation automatique, CloudWatch, Lambda, DynamoDB.

Abstract :

This internship, carried out at Smartovate LTD, consisted in designing and deploying an autonomous DevOps agent capable of automatically detecting and remediating incidents on an Amazon Web Services infrastructure. The solution relies on a CloudWatch → SNS → Lambda event chain : the alarm is diagnosed by a Lambda function (Amazon Bedrock with a fallback demo mode), classified into an incident category, and an appropriate remediation action is executed through AWS Systems Manager (service restart, Auto Scaling group scale-up, deployment rollback). An anti-flapping lock stored in DynamoDB with automatic expiration (TTL) protects the system against repeated executions, while all 39 resources are provisioned by Terraform following Infrastructure as Code principles. End-to-end tests validated the real scale-up remediation (up to 3 instances) and the anti-flapping blocking mechanism (HTTP 429).

Key-words : DevOps, AWS, Serverless, Terraform, Infrastructure as Code, Anti-flapping, Automated remediation, CloudWatch, Lambda, DynamoDB.